# Computational Work 3

Éderson Passos Silva Registration: 00318375

Course: Computational Matrix Methods

Professor: Cibele Aparecida Ladeia

**Abstract**

This work investigates two least-squares problems using data derived from the student's enrollment number. First, an overdetermined linear system $A\mathbf{x} \approx \mathbf{b}$, with $A \in \mathbb{R}^{5\times4}$, is solved using the QR factorization $A = QR$. The least-squares solution is obtained by solving the resulting upper triangular system. The existence of a unique solution is discussed in terms of the rank of the coefficient matrix.

Second, a quadratic polynomial is fitted to six experimental data points using the least-squares formulation. The associated design matrix is constructed, and the polynomial coefficients are determined computationally. The quality of the fit is assessed by plotting the experimental data and the fitted curve and calculating the mean squared error.

The numerical implementation explicitly constructs the principal linear algebra algorithms, avoiding high-level routines for QR factorization and least-squares solutions. This approach connects the mathematical foundations of least squares with their computational implementation and numerical evaluation.

**Keywords:** QR factorization; least squares; quadratic polynomial fitting; numerical linear algebra.

# 1 Introduction

Least-squares methods provide a systematic way to obtain an approximate solution when a linear system is overdetermined or when a set of observations cannot be represented exactly by a prescribed model. Their reliability depends not only on the mathematical formulation but also on the numerical properties of the algorithms used to compute the solution [1, 3].

This work investigates two least-squares problems specified in the Computational Work 3 assignment. First, a linear system $A\mathbf{x} \approx \mathbf{b}$, with $A \in \mathbb{R}^{5\times4}$, is addressed through the reduced QR factorization of $A$. Second, six data points are fitted with a polynomial of degree at most two, expressed as a linear least-squares problem involving a design matrix $X \in \mathbb{R}^{6\times3}$. In both cases, the data are constructed according to the assignment instructions from the digits of the student's enrollment number.

The computational objective is to connect these formulations to their implementation. The main numerical routines—including inner products, vector and matrix operations, the Modified Gram–Schmidt QR factorization, backward substitution, and error indicators—are implemented explicitly in Python. NumPy is retained for the array construction required by the assignment and for generating plotting coordinates, while Matplotlib is used to visualize the polynomial fit. No high-level NumPy or SciPy solver is used to calculate the QR factorization or the least-squares coefficients. The implementation and selected code excerpts are provided in the appendix.

The computed solutions are assessed through the QR reconstruction error, the orthogonality error of $Q$, the least-squares optimality condition, and, for the polynomial fit, the mean squared error. These measures serve different purposes: the first indicators assess numerical consistency of the algorithms, whereas the mean squared error quantifies the discrepancy between the observations and the fitted model. The analysis also examines the conditions under which each least-squares solution is unique, while acknowledging the numerical limitations of Modified Gram–Schmidt without column pivoting [2].

# 2 Problem Formulation

This work addresses two computational problems involving least-squares methods. The numerical data are constructed from the eight digits of the student's enrollment number. Any zero digit is replaced by one before constructing the matrices and vectors.

## 2.1 Problem 1: QR Factorization and Linear Least Squares

Consider the overdetermined linear system

$$A\mathbf{x} \approx \mathbf{b}, \tag{1}$$

where $A \in \mathbb{R}^{5\times 4}$, $\mathbf{x} \in \mathbb{R}^4$, and $\mathbf{b} \in \mathbb{R}^5$. The coefficient matrix and right-hand-side vector are defined by

$$A = \begin{bmatrix} m_1+2 & m_2 & -m_3 & m_4 \\ m_2+1 & -m_3 & m_4+2 & m_5 \\ m_3+2 & -m_4 & m_5+1 & -m_6 \\ -m_4 & m_5+2 & m_6 & m_7 \\ -m_5 & m_6+1 & m_7+2 & m_8 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} m_1+m_2 \\ -m_3 \\ m_4+m_5 \\ -m_6 \\ m_7+m_8 \end{bmatrix}. \tag{2}$$

The first objective is to compute the reduced QR factorization

$$A = QR, \tag{3}$$

where $Q \in \mathbb{R}^{5\times 4}$ has orthonormal columns and $R \in \mathbb{R}^{4\times 4}$ is upper triangular.

The least-squares solution is defined as

$$\mathbf{x}^* = \arg\min_{\mathbf{x}\in\mathbb{R}^4} \|A\mathbf{x} - \mathbf{b}\|_2^2. \tag{4}$$

The uniqueness of this solution depends on the column rank of $A$. In particular, the solution is unique if and only if $\text{rank}(A) = 4$. This condition will be assessed computationally for the given enrollment number.

## 2.2 Problem 2: Quadratic Least-Squares Fitting

Consider six experimental data points $(x_i, y_i)$, where

$$x_i = i, \qquad y_i = m_i \bmod 5, \qquad i = 1, \ldots, 6. \tag{5}$$

The objective is to approximate these data using a quadratic polynomial

$$p(x) = ax^2 + bx + c. \tag{6}$$

Defining the coefficient vector $\boldsymbol{\theta} = [a, b, c]^T$, the fitting problem can be expressed as

$$X\boldsymbol{\theta} \approx \mathbf{y}, \tag{7}$$

where

$$X = \begin{bmatrix} 1^2 & 1 & 1 \\ 2^2 & 2 & 1 \\ 3^2 & 3 & 1 \\ 4^2 & 4 & 1 \\ 5^2 & 5 & 1 \\ 6^2 & 6 & 1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \end{bmatrix}. \tag{8}$$

The polynomial coefficients are obtained by solving

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta} \in \mathbb{R}^3} \|X\boldsymbol{\theta} - \mathbf{y}\|_2^2. \tag{9}$$

Since the six values of $x_i$ are distinct, the three columns of $X$ are linearly independent. Therefore, $\text{rank}(X) = 3$, and the least-squares solution is unique.

Finally, the quality of the fitted polynomial is evaluated by plotting the experimental points and the fitted curve and by computing the mean squared error (MSE):

$$\text{MSE} = \frac{1}{6} \sum_{i=1}^{6} \left[ y_i - p(x_i) \right]^2. \tag{10}$$

# 3 Methodology

## 3.1 Data Construction

The numerical data are generated according to the instructions provided in the assignment. The student's enrollment number is converted into eight integer digits, replacing every zero with one. These digits are then used to construct the coefficient matrix $A$, the vector **b**, and the experimental observations $(x_i, y_i)$.

The original data-generation expressions are preserved. NumPy is used for the array construction specified in the assignment, whereas the principal linear algebra operations are implemented explicitly using Python loops. In particular, no predefined routines for QR factorization, least-squares solutions,

or polynomial fitting are used.

## 3.2 QR Factorization by Modified Gram–Schmidt

Let $B \in \mathbb{R}^{m\times n}$, with $m \geq n$, be a matrix with linearly independent columns. The QR factorization expresses $B$ as

$$B = QR, \tag{11}$$

where $Q \in \mathbb{R}^{m\times n}$ has orthonormal columns, $Q^TQ = I_n$, and $R \in \mathbb{R}^{n\times n}$ is upper triangular.

The factorization is computed using the modified Gram–Schmidt (MGS) algorithm. This method constructs an orthonormal basis for the column space of $B$ by successively removing the components of each column in the directions of previously computed basis vectors.

Denote the $j$-th column of $B$ by $\mathbf{b}_j$. Initialize

$$\mathbf{v}_j^{(0)} = \mathbf{b}_j. \tag{12}$$

For $i = 1, \ldots, j-1$, compute

$$r_{ij} = \mathbf{q}_i^T \mathbf{v}_j^{(i-1)}, \tag{13}$$
$$\mathbf{v}_j^{(i)} = \mathbf{v}_j^{(i-1)} - r_{ij}\mathbf{q}_i. \tag{14}$$

The next diagonal entry of $R$ and the corresponding column of $Q$ are then obtained as

$$r_{jj} = \|\mathbf{v}_j^{(j-1)}\|_2, \tag{15}$$
$$\mathbf{q}_j = \frac{\mathbf{v}_j^{(j-1)}}{r_{jj}}. \tag{16}$$

The algorithm is applied first to the matrix $A$ and subsequently to the design matrix $X$. The resulting factors are used to solve both least-squares problems. During implementation, a diagonal entry of $R$ that is numerically negligible is treated as an indication of possible rank deficiency; the corresponding case must be reported rather than dividing by that entry.

## 3.3 Selection of the Orthogonalization Method

Several methods can compute the QR factorization. The choice depends on numerical stability, implementation complexity, and the objectives of the computational assignment.

The modified Gram–Schmidt method is selected primarily for its transparent computational structure and suitability for a from-scratch implementation. It provides a direct illustration of orthogonalization while improving numerical behavior relative to classical Gram–Schmidt. Householder QR would generally be preferable when numerical robustness is the primary concern, particularly for larger or ill-conditioned problems [1, 3, 2].

The choice is therefore pedagogical and computational rather than a claim that modified Gram–Schmidt is universally the most stable QR algorithm.

Table 1: Comparison of methods for computing the QR factorization.

| Method | Advantages | Limitations |
|---|---|---|
| Classical Gram–Schmidt | Simple formulation and straightforward implementation. | Can lose orthogonality when the columns are nearly linearly dependent. |
| Modified Gram–Schmidt | Explicit construction of the orthonormal basis; generally preserves orthogonality better than the classical variant and is convenient for implementing the algorithm from first principles. | Can still lose orthogonality for ill-conditioned matrices; it is not generally as numerically robust as Householder QR. |
| Householder reflections | Highly reliable numerical behavior and a standard choice for dense QR factorizations. | Requires implementing and managing reflection transformations, making the algorithm less direct to explain from first principles. |
| Givens rotations | Useful for exploiting sparsity and updating factorizations when individual rows or entries change. | Requires managing a sequence of rotations, which is unnecessary for the small dense matrices considered here. |

## 3.4 Least-Squares Solution Using QR

Consider the overdetermined system

$$B\mathbf{z} \approx \mathbf{c}, \tag{17}$$

where $B$ has full column rank. The least-squares solution minimizes the squared Euclidean residual:

$$\mathbf{z}^* = \arg\min_{\mathbf{z}\in\mathbb{R}^n} \|B\mathbf{z} - \mathbf{c}\|_2^2. \tag{18}$$

Substituting $B = QR$ and using the orthonormality of the columns of $Q$ gives

$$\begin{aligned} \|B\mathbf{z} - \mathbf{c}\|_2^2 = \|R\mathbf{z} - Q^T\mathbf{c}\|_2^2 \\ + \|(I - QQ^T)\mathbf{c}\|_2^2. \end{aligned} \tag{19}$$

The second term is independent of $\mathbf{z}$. Consequently, the least-squares solution is obtained by solving the upper triangular system

$$R\mathbf{z} = Q^T\mathbf{c}. \tag{20}$$

This system is solved by back substitution, starting from the last equation:

$$z_i = \frac{d_i - \sum_{j=i+1}^{n} R_{ij} z_j}{R_{ii}}, \qquad i = n, n-1, \ldots, 1, \tag{21}$$

where $\mathbf{d} = Q^T\mathbf{c}$.

For the first problem, $B = A$, $\mathbf{c} = \mathbf{b}$, and $\mathbf{z} = \mathbf{x}$. For the second problem, $B = X$, $\mathbf{c} = \mathbf{y}$, and $\mathbf{z} = [a, b, c]^T$.

The QR-based approach is preferred to solving the normal equations $B^TB\mathbf{z} = B^T\mathbf{c}$. For a full-

column-rank matrix, the normal equations satisfy $\kappa_2(B^TB) = \kappa_2(B)^2$, potentially amplifying the effects of poor conditioning. The QR formulation avoids explicitly forming $B^TB$ and reduces the least-squares problem to a triangular solve [1, 2].

### 3.5 Numerical Verification and Error Evaluation

The computed QR factors are assessed using two quantities. The relative factorization error is defined as

$$\eta_{\text{QR}} = \frac{\|B - QR\|_F}{\|B\|_F}, \tag{22}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. The orthogonality error is defined as

$$\eta_Q = \|Q^TQ - I_n\|_F. \tag{23}$$

Both quantities are evaluated using explicitly implemented matrix operations.

For the first problem, the residual norm

$$\rho_A = \|A\mathbf{x}^* - \mathbf{b}\|_2 \tag{24}$$

is calculated to assess the least-squares solution. The uniqueness of the solution is examined through the column rank of $A$ and the diagonal entries of $R$.

For the quadratic fit, the predicted values are calculated from

$$\widehat{y}_i = ax_i^2 + bx_i + c, \tag{25}$$

and the mean squared error is computed as

$$\text{MSE} = \frac{1}{6}\sum_{i=1}^{6}(y_i - \widehat{y}_i)^2. \tag{26}$$

Finally, the experimental observations and the fitted polynomial are plotted together. The numerical algorithms are implemented without predefined linear algebra or polynomial-fitting routines; a plotting library is used only to visualize the results.

The complete computational implementation, including the data-generation code and the routines for QR factorization, back substitution, error evaluation, and plotting, is provided in the appendix.

## 4 Results Discussion

### 4.1 QR Factorization and Linear Least-Squares Solution

The modified Gram–Schmidt algorithm was applied to the overdetermined system defined in Section 2. The computed factors satisfy $A \approx QR$, where $Q \in \mathbb{R}^{5\times4}$ has approximately orthonormal columns and $R \in \mathbb{R}^{4\times4}$ is upper triangular.

The computed matrices, rounded to six decimal places, are

$$Q \approx \begin{bmatrix} 0.295599 & 0.242348 & -0.209363 & 0.159387 \\ 0.197066 & -0.200200 & 0.322900 & 0.898629 \\ 0.492665 & 0.140811 & 0.781674 & -0.352688 \\ -0.098533 & 0.938737 & -0.000348 & 0.202929 \\ -0.788263 & 0.011495 & 0.490803 & 0.038631 \end{bmatrix}, \tag{27}$$

and

$$R \approx \begin{bmatrix} 10.148892 & -4.926646 & -3.251587 & -4.236916 \\ 0 & 10.135490 & 2.859317 & 4.846950 \\ 0 & 0 & 13.048045 & 2.480397 \\ 0 & 0 & 0 & 10.020142 \end{bmatrix}. \tag{28}$$

The quality of the factorization was assessed using the relative reconstruction error and the orthogonality error. The computed values were

$$\eta_{\text{QR}} = 1.0683 \times 10^{-16}, \qquad \eta_Q = 4.5180 \times 10^{-16}. \tag{29}$$

Both indicators are close to the expected scale of floating-point rounding errors for this computation. These results support the numerical accuracy of the factorization and the orthogonality of the computed columns of $Q$.

The least-squares solution was obtained by solving $R\mathbf{x}^* = Q^T\mathbf{b}$ using back substitution. The resulting solution, rounded to nine decimal places, is

$$\mathbf{x}^* \approx \begin{bmatrix} -0.404938762 \\ -0.040220457 \\ 0.992366790 \\ -0.568506366 \end{bmatrix}. \tag{30}$$

The Euclidean norm of the residual was

$$\|\mathbf{A}\mathbf{x}^* - \mathbf{b}\|_2 \approx 7.677354. \tag{31}$$

The nonzero residual indicates that the original system is inconsistent. This is expected for an overdetermined least-squares problem and does not, by itself, indicate a failure of the numerical method. Instead, the solution minimizes the residual norm among all vectors in $\mathbb{R}^4$.

To assess the optimality of the computed solution, the norm of the residual projected onto the column directions of $A$ was also evaluated. The result was

$$\left\|A^T(A\mathbf{x}^* - \mathbf{b})\right\|_2 \approx 6.1020 \times 10^{-14}. \tag{32}$$

This value is numerically small and is consistent with the least-squares optimality condition $A^T(A\mathbf{x}^* - \mathbf{b}) = \mathbf{0}$. Hence, the computed residual is approximately orthogonal to the column space of $A$.

Finally, the diagonal entries of $R$ were compared with the scale-dependent numerical tolerance used by the implementation. All four diagonal entries were substantially larger than this tolerance,

supporting the numerical assessment that $A$ has full column rank. Therefore, the least-squares solution is unique.

These results validate the use of the modified Gram–Schmidt factorization and back substitution for the matrix considered in this experiment. Nevertheless, modified Gram–Schmidt without column pivoting is not a universally reliable rank-revealing method for severely ill-conditioned matrices. Householder QR or rank-revealing factorizations may be preferable when stronger numerical robustness is required [1, 2].

## 4.2 Quadratic Least-Squares Fitting

The second experiment considered the least-squares approximation of six observations by a polynomial of degree at most two. The design matrix $X \in \mathbb{R}^{6\times 3}$ was constructed using the columns associated with $x^2$, $x$, and the constant term. The coefficient vector $\boldsymbol{\theta} = [a, b, c]^T$ was computed using the reduced QR factorization and back substitution.

The computed coefficients were

$$\boldsymbol{\theta}^* \approx \begin{bmatrix} 1.9739 \times 10^{-15} \\ 0.4000 \\ 0.6000 \end{bmatrix}. \tag{33}$$

Thus, the fitted polynomial is

$$p(x) \approx 0.4000x + 0.6000, \tag{34}$$

because the estimated quadratic coefficient is numerically negligible. Although a quadratic model was specified, the resulting least-squares polynomial is effectively linear at the displayed precision. This behavior is a property of the data and does not indicate a failure of the numerical algorithm.

Table 2 compares the observations with the fitted values and the corresponding residuals.

Table 2: Observations, fitted values, and residuals.

| $x_i$ | $y_i$ | $\widehat{y}_i$ | $y_i - \widehat{y}_i$ |
|---|---|---|---|
| 1 | 1 | 1.0 | 0.0 |
| 2 | 1 | 1.4 | -0.4 |
| 3 | 3 | 1.8 | 1.2 |
| 4 | 1 | 2.2 | -1.2 |
| 5 | 3 | 2.6 | 0.4 |
| 6 | 3 | 3.0 | 0.0 |

The residual vector has Euclidean norm

$$\|\mathbf{y} - X\boldsymbol{\theta}^*\|_2 \approx 1.7889. \tag{35}$$

The mean squared error is

$$\text{MSE} = \frac{1}{6}\sum_{i=1}^{6}(y_i - \widehat{y}_i)^2 = \frac{3.2}{6} \approx 0.5333. \tag{36}$$

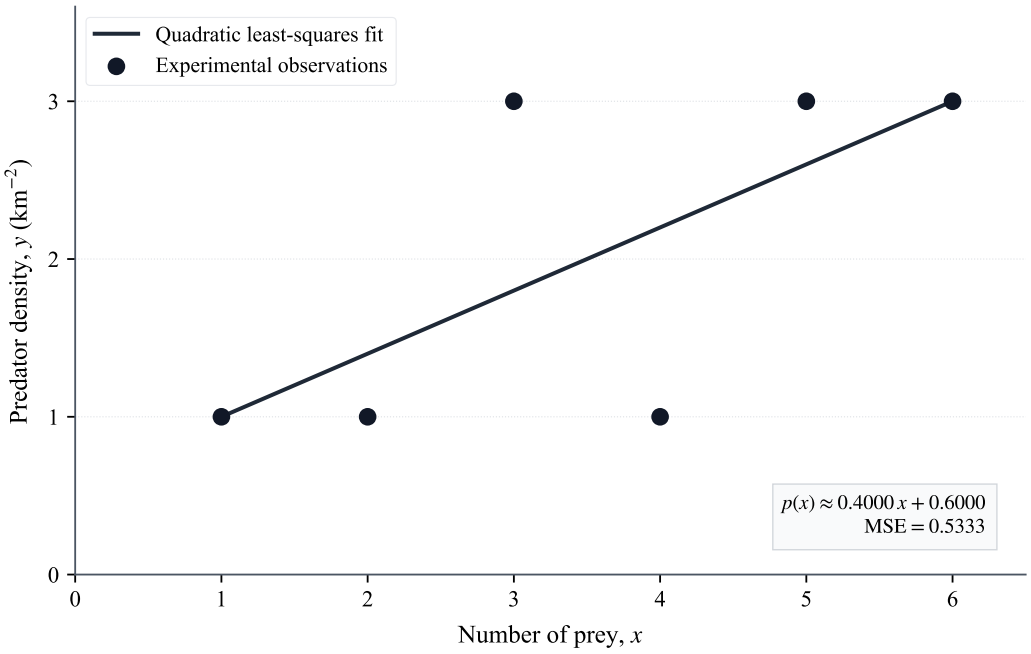
The residual is nonzero because the fitted polynomial does not pass through every observation. This is expected in a least-squares approximation and does not imply that the computed minimizer is inaccurate. The MSE quantifies the discrepancy between the data and the fitted model, whereas the numerical indicators assess the accuracy of the computational procedure.

The QR factorization produced a relative reconstruction error of $7.3017 \times 10^{-17}$, while the orthogonality error was $3.9468 \times 10^{-15}$. The least-squares optimality indicator, $\|X^T(X\boldsymbol{\theta}^* - \mathbf{y})\|_2$, was $2.3055 \times 10^{-14}$. These values are consistent with a numerically accurate solution for this problem.

The uniqueness of the solution follows from the full column rank of $X$. Indeed, if $X\boldsymbol{\theta} = \mathbf{0}$, then the polynomial $ax^2 + bx + c$ vanishes at the six distinct values $x_i = 1, \ldots, 6$. A nonzero polynomial of degree at most two cannot have six distinct roots. Therefore, $\boldsymbol{\theta} = \mathbf{0}$, so $\text{rank}(X) = 3$ and the least-squares minimizer is unique.

Figure 1 presents the observations and the fitted polynomial. The nearly straight fitted curve is consistent with the numerically negligible quadratic coefficient.

Figure 1: Experimental observations and the quadratic least-squares fitted curve.




Source: Elaborated by the author.

Since the observations are constructed from the enrollment digits as prescribed by the assignment, the analysis concerns the numerical behavior of the fitting procedure rather than an empirical conclusion about predator–prey dynamics.

# 5 Conclusion

This work implemented and evaluated two least-squares procedures using a reduced QR factorization computed by the Modified Gram–Schmidt method. The numerical routines for inner products, norms, matrix operations, QR factorization, and backward substitution were written explicitly in Python rather than delegated to high-level linear algebra solvers. The results demonstrate how the QR formulation

converts each overdetermined problem into an upper-triangular system while allowing the computation to be assessed through independent numerical indicators.

For the first problem, the relative QR reconstruction error was approximately $1.07 \times 10^{-16}$, and the orthogonality error was approximately $4.52 \times 10^{-16}$. The optimality indicator, $\|A^T(A\mathbf{x}^* - \mathbf{b})\|_2$, was approximately $6.10 \times 10^{-14}$. Together with the full-column-rank diagnostic, these values are consistent with a numerically accurate and unique least-squares solution for the matrix considered. The residual norm remained nonzero, as expected for an inconsistent overdetermined system; it measures the distance from $\mathbf{b}$ to the column space of $A$ rather than a failure of the algorithm.

For the second problem, the estimated coefficients were $a \approx 1.97 \times 10^{-15}$, $b = 0.4$, and $c = 0.6$. Thus, although a polynomial of degree at most two was prescribed, the fitted polynomial is effectively linear at the reported precision: $p(x) \approx 0.4x + 0.6$. The mean squared error was approximately $0.5333$, while the QR reconstruction, orthogonality, and least-squares optimality indicators were approximately $7.30 \times 10^{-17}$, $3.95 \times 10^{-15}$, and $2.31 \times 10^{-14}$, respectively. This illustrates the distinction between numerical accuracy and fit quality: small algorithmic errors do not imply that the model reproduces every observation exactly.

The full-column-rank condition ensures uniqueness in both problems. For the quadratic fit, this follows theoretically because the six distinct abscissas make the three columns of the design matrix linearly independent. Numerically, the diagonals of the computed triangular factors also exceeded the adopted rank tolerance. Nevertheless, Modified Gram–Schmidt without column pivoting is not a universally reliable rank-revealing method for severely ill-conditioned matrices; a pivoted QR factorization or singular value decomposition may be preferable when stronger rank diagnostics are required [2].

Finally, the observations in this assignment are generated from the enrollment digits according to the prescribed rules. Consequently, the second experiment illustrates polynomial least-squares fitting and numerical validation, not an empirical inference about predator–prey dynamics. Overall, the two experiments demonstrate the practical connection between least-squares theory, explicit algorithmic implementation, and numerical verification.

# References


[1] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, Baltimore, 4 edition, 2013.

[2] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition, 2002.

[3] Lloyd N. Trefethen and David Bau, III. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1997.

# A Computational Implementation

The core numerical algorithms were implemented explicitly with Python lists, loops, arithmetic operations, and the standard library. NumPy is used for the input-array construction specified in the assignment and to generate plotting coordinates; Matplotlib is used only to render the figure. The solution does not call high-level library solvers such as `numpy.linalg.qr`, `numpy.linalg.lstsq`, `numpy.linalg.solve`, or `numpy.polyfit`. These abridged excerpts rely on helper routines also defined in the source modules; the complete executable files include validation, diagnostics, command-line interaction, and report formatting.

## A.1 QR Factorization and Least-Squares Solution

The inner products, norms, vector updates, and matrix operations used by the solver are implemented with loops and summations. The reduced QR factorization uses Modified Gram–Schmidt: each column is successively orthogonalized against the previously computed columns of $Q$. The projection coefficients are stored in the upper-triangular matrix $R$.

Listing 1: Core operations and Modified Gram–Schmidt factorization from `linear_algebra.py`.

```
import math
import sys


def vector_dot(u, v):
    return sum(ui * vi for ui, vi in zip(u, v))


def vector_norm(u):
    return math.sqrt(sum(ui * ui for ui in u))


def qr_mgs(A, tol=None):
    A = to_float_matrix(A)
    m, n = matrix_shape(A)
    scale = frobenius_norm(A)
    if tol is None:
        tol = max(m, n) * sys.float_info.epsilon * scale

    Q = [[0.0 for _ in range(n)] for _ in range(m)]
    R = [[0.0 for _ in range(n)] for _ in range(n)]
    q_cols = []
    full_rank = True

    for j in range(n):
        v = get_column(A, j)
        for i in range(j):
            qi = q_cols[i]
            R[i][j] = vector_dot(qi, v)
            v = [v[k] - R[i][j] * qi[k] for k in range(m)]

        R[j][j] = vector_norm(v)
        if R[j][j] <= tol:
            full_rank = False
            qj = [0.0] * m
        else:
            qj = [v[k] / R[j][j] for k in range(m)]
        q_cols.append(qj)
```

```
        for k in range(m):
            Q[k][j] = qj[k]

    info = {
        "is_full_rank": full_rank,
        "tolerance": tol,
        "diagonal_entries": [R[i][i] for i in range(n)],
    }
    return Q, R, info
```

The rank threshold is scale-dependent,

$$\text{tol} = \max(m, n)\, \epsilon_{\text{mach}}\, \|A\|_F.$$

If a diagonal entry of $R$ does not exceed this threshold, the implementation flags possible numerical rank deficiency and avoids normalizing by a negligible value. Modified Gram–Schmidt without column pivoting is not a universally reliable rank-revealing method for severely ill-conditioned matrices.

For full-column-rank matrices, $A = QR$ reduces the least-squares problem to $R\mathbf{x} = Q^T\mathbf{b}$. The program forms $Q^T\mathbf{b}$ with its own matrix–vector routine and solves the triangular system by backward substitution.

Listing 2: Back substitution and the QR-based least-squares solve.

```
def back_substitution(R, d, tol):
    n = len(R)
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        if abs(R[i][i]) <= tol:
            raise RankDeficiencyError("Numerically singular system")
        s = sum(R[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (d[i] - s) / R[i][i]
    return x


def solve_least_squares_qr(A, b, tol=None):
    Q, R, info = qr_mgs(A, tol)
    if not info["is_full_rank"]:
        raise RankDeficiencyError("Possible rank deficiency")
    d = matrix_vector_mul(matrix_transpose(Q), b)
    x = back_substitution(R, d, info["tolerance"])
    return x, Q, R, info
```

## A.2 Data Construction and Quadratic Fit

For Problem 1, the matrix and right-hand-side vector retain the expressions and NumPy array construction supplied in the assignment. That prescribed constructor is not reproduced here. For Problem 2, the design matrix is assembled as rows $(x_i^2, x_i, 1)$ using Python list operations; the same QR solver is reused, and the MSE is calculated directly from the residuals.

Listing 3: Problem 2: design matrix, fitted values, and explicit MSE.

```
# x and y are generated according to the assignment.
X = [[float(xi)**2, float(xi), 1.0] for xi in x]
theta, Q, R, info = solve_least_squares_qr(X, y)
a, b_coef, c = theta
y_hat = [a*xi**2 + b_coef*xi + c for xi in x]
```

```
mse = sum((yi - pi)**2 for yi, pi in zip(y, y_hat)) / len(y)
```

## A.3 Numerical Verification and Visualization

The verification metrics are computed using the same explicit matrix and vector routines. For each matrix $B$ and right-hand side $\mathbf{c}$, the implementation evaluates

$$\eta_{\text{QR}} = \frac{\|B - QR\|_F}{\|B\|_F}, \qquad \eta_Q = \|Q^T Q - I\|_F, \tag{37}$$

$$\rho = \|B\mathbf{z}^* - \mathbf{c}\|_2, \qquad \eta_{\text{LS}} = \|B^T(B\mathbf{z}^* - \mathbf{c})\|_2. \tag{38}$$

Here, $\eta_{\text{QR}}$ and $\eta_Q$ assess reconstruction and orthogonality, whereas $\eta_{\text{LS}}$ checks the least-squares optimality condition. The residual norm and MSE measure fit to the data, not numerical error in the solver. Matplotlib is used only to render the curve and observations; it does not estimate the polynomial coefficients.